SupportPilot: Technical Implementation Guide

From an empty folder to a deployed demo. Every command, every file, in the order you should do them. A ready-made project folder (**supportpilot.zip**) contains all code shown here, so you can either type along or unzip it.

> **What was and wasn't tested**
> The pure-Python parts were run and checked: dataset builder, train/eval leak check, JSON validation, metric code, chunking, notebook JSON. I could **not** run Qwen, FAISS, Streamlit, Kaggle or Supabase end-to-end from here (no model downloads possible), so expect to fix small version-related errors on your machine. Section 12 lists the likely ones and their fixes.

0. What you are building (the whole picture)

```
Customer message
      |
      v
 [Streamlit app] --> [Retrieval: MiniLM embeddings + FAISS] --> top passages (+ similarity scores)
      |                                                              |
      |        evidence below MIN_SCORE is dropped; none left => forced escalation
      v                                                              v
 [Qwen2.5-1.5B + your LoRA adapter]  <---- same SYSTEM prompt + "Customer message / Relevant
   documentation"
      |
      v
 JSON {category, priority, needs_human, draft_reply, reason} --> validated in code --> inbox UI
      |                                  (malformed output => safe fallback + escalate)
      v
 operator edits draft, clicks Useful / Incorrect / Needs escalation --> data/feedback.jsonl
```

Where	What runs there	Why
Your MacBook	Coding, dataset building, retrieval, baseline + fine-tuned evaluation, the Streamlit app	Free, fast iteration
Kaggle notebook	LoRA training only	Free GPU; your Mac never trains
GitHub	Source code (no weights, no secrets)	Version control, deploy source
Supabase (later)	Users, workspaces, tickets, drafts, feedback	Auth + Postgres with row-level security
Tunnel / HF Space	Public demo	Free ways to show it to prospects

Order of work: accounts and tools → folders → dataset → baseline score → Kaggle training → fine-tuned score → app → (Supabase) → deploy. Do not skip the baseline: it is how you find out whether fine-tuning helped at all.

1. Accounts and tools to set up first

Item	Needed for	Notes
GitHub account	Code hosting	Free

Item	Needed for	Notes
Kaggle account	GPU training	Verify your phone number: this unlocks GPU and Internet in notebooks. GPU hours are weekly-limited; check your quota on your Kaggle account page.
Hugging Face account	Model download, optional Space	Qwen2.5 is public, so no token is needed just to download it. A token is needed to upload.
Supabase account	Phase 9 only	Create it later, once the core workflow works
Homebrew, Python 3.11, git, gh, VS Code	Your Mac	Commands below

Open **Terminal** on the Mac and run:

```
# 1. Apple command-line tools (git, compilers)
xcode-select --install

# 2. Homebrew (package manager) - skip if `brew --version` already works
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 3. Tools
brew install python@3.11 git gh cloudflared
brew install --cask visual-studio-code      # optional editor

# 4. Check
python3.11 --version
git --version
```

2. Create the project folders

Option A: unzip the starter project and skip to the virtual-environment commands.

```
mkdir -p ~/projects && cd ~/projects
unzip ~/Downloads/supportpilot.zip          # creates ~/projects/supportpilot
cd supportpilot
```

Option B: build the structure yourself.

```
mkdir -p ~/projects/supportpilot && cd ~/projects/supportpilot
mkdir -p data/demo_docs notebooks models src scripts results supabase deploy
touch src/__init__.py scripts/__init__.py models/.gitkeep
code .          # opens VS Code here
```

Then, in both cases, create the isolated Python environment and install the app dependencies:

```
python3.11 -m venv .venv
source .venv/bin/activate            # your prompt now starts with (.venv); repeat this in every new
  terminal
python -m pip install --upgrade pip
pip install -r requirements.txt      # ~2-3 GB of downloads (PyTorch etc.)

# confirm Apple GPU (MPS) is visible to PyTorch
python -c "import torch; print(torch.__version__, 'MPS:', torch.backends.mps.is_available())"
```

Final layout:

```
supportpilot/
├── app.py                       Streamlit inbox
├── requirements.txt             app dependencies (Mac + deployment)
├── requirements-train.txt       Kaggle-only dependencies
├── .gitignore
├── data/
│   ├── train.jsonl  eval.jsonl         (generated by scripts/make_dataset.py)
│   └── demo_docs/acme_faq.md           (generated, fictional help center)
├── notebooks/finetune_support.ipynb    Kaggle training notebook
├── models/supportpilot-lora/           adapter you download from Kaggle (git-ignored)
├── results/                            evaluation outputs (git-ignored)
├── scripts/  make_dataset.py  validate_dataset.py  run_eval.py  compare.py  scenarios.py
├── src/      prompts.py  knowledge_base.py  assistant.py  evaluation.py  db.py
├── supabase/schema.sql
└── deploy/   Dockerfile  space_README_header.md
```

Initialise git now, so every later step is saved:

```
git init -b main
git add .
git status                      # check that .venv/, models/ weights and .env are NOT listed
git commit -m "Initial project structure"
```

.gitignore and requirements

```
.venv/
__pycache__/
*.pyc
.env
.streamlit/secrets.toml
data/feedback.jsonl
results/
models/*
!models/.gitkeep
*.safetensors
*.bin
.DS_Store
```

```
streamlit>=1.35
pypdf>=4.0
sentence-transformers>=3.0
faiss-cpu>=1.8
numpy
torch
transformers>=4.45
accelerate
peft>=0.12
```

```
# Only for the Kaggle notebook (installed there with !pip). Not needed on your Mac.
transformers
datasets
peft
trl
accelerate
```

3. The shared prompt contract: src/prompts.py

The most common reason a fine-tune looks great in training and fails in the app is a **prompt mismatch**. So the system prompt and the user-message format live in exactly one file, imported by the dataset builder, the evaluator and the app. The Kaggle notebook never builds prompts itself; it only trains on the JSONL files that were built from this file.

```
"""Single source of truth for the prompt format.
The dataset builder, the Kaggle notebook, the evaluator and the app must all use THIS format,
otherwise the fine-tuned model sees a different prompt at inference than it saw in training."""

SYSTEM_PROMPT = (
    "You are a customer support triage assistant. Return only valid JSON with category, "
    "priority, needs_human, draft_reply, and reason. Never invent company policies. "
    "If evidence is missing, escalate."
)

CATEGORIES = ["billing", "technical_issue", "account_access", "feature_request", "how_to", "other"]
PRIORITIES = ["low", "medium", "high"]
NO_DOCS = "No relevant documentation found."


def build_user_message(question: str, docs_text: str) -> str:
    return f"Customer message: {question}\nRelevant documentation: {docs_text.strip() or NO_DOCS}"
```

4. Build the dataset

4.1 How the generator works

scripts/scenarios.py holds *scenarios*: one underlying situation (e.g. "duplicate charge") with several customer phrasings, a gold answer, and the documentation sentence that supports it (doc=None means the docs have no answer, so the correct behaviour is to escalate).
scripts/make_dataset.py turns each phrasing into several training rows:

- **Clean evidence:** only the correct doc sentence is shown.
- **Noisy retrieval:** the correct doc plus 1-2 unrelated doc sentences, shuffled, imitating a real retriever.
- **Answer not in the docs:** only unrelated docs are shown, so the label becomes escalate with a no-evidence reply. This is what teaches the model not to answer from imagination.
- **Unanswerable scenarios** are shown with either "No relevant documentation found." or unrelated docs.

The split is by **scenario**: every 4th scenario (sorted by id) is held out for eval, so paraphrases of a training scenario can never appear in eval. **validate_dataset.py** verifies the format and fails if any customer message appears in both files.

```
source .venv/bin/activate
python -m scripts.make_dataset
python -m scripts.validate_dataset data/train.jsonl data/eval.jsonl
wc -l data/*.jsonl
head -n 1 data/train.jsonl | python -m json.tool | head -30     # eyeball one row

# Expected with the 19 seed scenarios:  train examples: 126   eval examples: 42
```

> **Be honest about this seed set**
> 19 scenarios is a pipeline smoke test, not a product dataset, and 42 eval rows is too few to trust small differences. The generator produces roughly 8-9 rows per scenario, so **~60 scenarios gives ~500 rows and ~100 gives ~900**. Held-out eval should have 100+ rows, i.e. 12+ held-out scenarios, so grow the list before you draw conclusions.

4.2 Growing the scenario list

Add scenarios by hand (best quality) and/or ask an LLM to draft them, then **read every one**. Paste this prompt into Claude or ChatGPT, together with your fictional company's help-center text:

```
Write 15 new support scenarios as Python dicts for a fictional SaaS company, in exactly this format:
dict(id="snake_case_id", category=<billing|technical_issue|account_access|feature_request|how_to|other>,
     priority=<low|medium|high>, needs_human=<True|False>,
     doc="<1-2 sentence help-center fact that answers it, or None if the docs have no answer>",
     questions=[<3-4 different customer phrasings: terse, polite, frustrated, typo-ridden>],
     reply="<short, polite draft reply using ONLY the doc>", reason="<one-sentence justification>")
Rules: refunds, account changes, security incidents, legal/privacy requests, and anything the doc
doesn't cover must have needs_human=True. Never invent policies. Include 3 prompt-injection attempts
(customer text tries to override instructions) and 3 vague/ambiguous messages.
```

Paste the results into the SCENARIOS list, keep ids unique, then re-run the three commands above. Never train on real customer conversations without permission.

4.3 scripts/scenarios.py (first 3 of 19 shown; the full file is in the zip)

```
"""Seed scenarios for the fictional company 'Acme Cloud'.
One scenario = one underlying situation, several customer phrasings.
doc=None means 'the documentation has no answer' (the model must escalate).
EXPAND THIS LIST to ~100+ scenarios before training for real (see the guide)."""

SCENARIOS = [
    dict(id="invite_member", category="how_to", priority="low", needs_human=False,
         doc="Workspace admins can invite teammates from Settings > Members by selecting Invite
           member.",
         questions=["How do I invite my teammate?", "need to add a new person to our workspace, where do
           i do that",
                    "Can I add my colleague to our account?", "Hi, how can I get my coworker access to
                      our projects?"],
         reply="A workspace admin can invite a teammate from Settings > Members by selecting Invite
           member.",
         reason="The documentation directly explains this process."),
    dict(id="reset_password", category="account_access", priority="medium", needs_human=False,
         doc="Users can reset their password from the login page by selecting Forgot password. A reset
           link is emailed and expires after 30 minutes.",
         questions=["I can't reset my password.", "forgot my password, how do I get back in?",
                    "Password reset please - I can't log in", "I never got into my account after
                      changing phones, how do I reset my login?"],
         reply="You can reset your password from the login page by selecting Forgot password. We'll
           email you a reset link, which expires after 30 minutes.",
         reason="The documentation covers the password reset process."),
    dict(id="update_card", category="billing", priority="low", needs_human=False,
         doc="Billing details can be updated in Settings > Billing > Payment method. Only workspace
           owners can change billing details.",
         questions=["How do I update my credit card?", "my card expired, where do I put the new one",
                    "Where can I change my billing information?"],
         reply="A workspace owner can update billing details in Settings > Billing > Payment method.",
         reason="The documentation explains where billing details are updated."),
    dict(id="duplicate_charge", category="billing", priority="medium", needs_human=True,
         doc="Billing disputes require review by the billing team. The assistant cannot issue refunds.",
```

4.4 scripts/make_dataset.py

```
"""python -m scripts.make_dataset
Builds data/train.jsonl, data/eval.jsonl and data/demo_docs/acme_faq.md from scripts/scenarios.py.
The split is BY SCENARIO (not by row) so paraphrases never leak from train into eval."""
import json
import random
from pathlib import Path

from src.prompts import NO_DOCS, SYSTEM_PROMPT, build_user_message
from .scenarios import SCENARIOS

random.seed(7)
NO_EVIDENCE_REPLY = ("I don't have verified information to answer this yet. "
                     "I'll flag it for the support team so they can follow up.")
NO_EVIDENCE_REASON = "The available documentation does not answer the question."


def example(question, docs_text, answer):
    return {"messages": [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": build_user_message(question, docs_text)},
        {"role": "assistant", "content": json.dumps(answer, ensure_ascii=False)},
    ]}


def answer_of(s):
    return {"category": s["category"], "priority": s["priority"], "needs_human": s["needs_human"],
            "draft_reply": s["reply"], "reason": s["reason"]}


def distractors(current_id, k):
    pool = [s["doc"] for s in SCENARIOS if s["doc"] and s["id"] != current_id]
    return random.sample(pool, k)


def join(docs):
    docs = docs[:]
    random.shuffle(docs)
    return "\n".join(docs)


def build(scenarios):
    rows = []
    for s in scenarios:
        for q in s["questions"]:
            if s["doc"]:
                rows.append(example(q, s["doc"], answer_of(s)))                                    #
                  clean evidence
                rows.append(example(q, join([s["doc"]] + distractors(s["id"], random.choice([1, 2]))),
                  answer_of(s)))  # noisy retrieval
                miss = {**answer_of(s), "needs_human": True, "draft_reply": NO_EVIDENCE_REPLY, "reason":
                  NO_EVIDENCE_REASON}
                rows.append(example(q, join(distractors(s["id"], random.choice([1, 2]))), miss))  #
                  answer NOT in retrieved docs
            else:
                rows.append(example(q, NO_DOCS, answer_of(s)))
                rows.append(example(q, join(distractors(s["id"], random.choice([1, 2]))), answer_of(s)))
    random.shuffle(rows)
    return rows
```

```
def write_jsonl(path, rows):
    with open(path, "w", encoding="utf-8") as f:
        for r in rows:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")


def main():
    ids = sorted(s["id"] for s in SCENARIOS)
    eval_ids = {sid for i, sid in enumerate(ids) if i % 4 == 3}      # every 4th scenario is held out
    train = build([s for s in SCENARIOS if s["id"] not in eval_ids])
    ev = build([s for s in SCENARIOS if s["id"] in eval_ids])
    Path("data").mkdir(exist_ok=True)
    write_jsonl("data/train.jsonl", train)
    write_jsonl("data/eval.jsonl", ev)

    faq = ["# Acme Cloud Help Center (fictional demo document)\n"]
    for s in SCENARIOS:
        if s["doc"]:
            faq.append(f"## {s['questions'][0]}\n{s['doc']}\n")
    Path("data/demo_docs").mkdir(parents=True, exist_ok=True)
    Path("data/demo_docs/acme_faq.md").write_text("\n".join(faq), encoding="utf-8")

    print(f"scenarios: {len(SCENARIOS)}  held-out for eval: {sorted(eval_ids)}")
    print(f"train examples: {len(train)}   eval examples: {len(ev)}")


if __name__ == "__main__":
    main()
```

4.5 scripts/validate_dataset.py

```
"""python -m scripts.validate_dataset data/train.jsonl data/eval.jsonl
Checks every line is valid chat JSON with a valid answer, and that no user message appears in both
  files."""
import json
import sys

from src.assistant import parse_output


def load(path):
    rows = []
    for n, line in enumerate(open(path, encoding="utf-8"), 1):
        row = json.loads(line)
        roles = [m["role"] for m in row["messages"]]
        assert roles == ["system", "user", "assistant"], f"{path}:{n} bad roles {roles}"
        _, ok = parse_output(row["messages"][2]["content"])
        assert ok, f"{path}:{n} assistant answer is not valid"
        rows.append(row)
    return rows


train, ev = load(sys.argv[1]), load(sys.argv[2])
q = lambda rows: {r["messages"][1]["content"].split("\n")[0] for r in rows}
overlap = q(train) & q(ev)
print(f"train={len(train)} eval={len(ev)} overlapping customer messages={len(overlap)}")
assert not overlap, f"LEAK: {sorted(overlap)[:3]}"
print("OK")
```

5. Knowledge base (retrieval): src/knowledge_base.py

Flow: read file → split into 100-word chunks with 20 words of overlap → embed each chunk with all-MiniLM-L6-v2 → store in a FAISS inner-product index (vectors are normalised, so inner product = cosine similarity) → at question time embed the question and return the top-k chunks with their scores.

```
import io
from functools import lru_cache
from pathlib import Path

import faiss
import numpy as np
from pypdf import PdfReader


@lru_cache(maxsize=1)
def get_embedder():
    from sentence_transformers import SentenceTransformer
    return SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")


def read_document(filename: str, content: bytes) -> str:
    suffix = Path(filename).suffix.lower()
    if suffix == ".pdf":
        reader = PdfReader(io.BytesIO(content))
        return "\n".join(page.extract_text() or "" for page in reader.pages)
    if suffix in {".txt", ".md"}:
        return content.decode("utf-8", errors="replace")
    raise ValueError("Only PDF, TXT, and Markdown are supported.")


def chunk_text(text: str, size: int = 100, overlap: int = 20):
    words = text.split()
    chunks = []
    for start in range(0, len(words), size - overlap):
        chunk = " ".join(words[start:start + size])
        if chunk.strip():
            chunks.append(chunk)
    return chunks


def build_index(documents):
    """documents: list of {"filename": str, "text": str}"""
    chunks = []
    for doc in documents:
        for i, chunk in enumerate(chunk_text(doc["text"])):
            chunks.append({"text": chunk, "source": doc["filename"], "chunk": i})
    if not chunks:
        raise ValueError("No readable text found in uploaded documents.")
    vectors = get_embedder().encode(
        [c["text"] for c in chunks], normalize_embeddings=True, convert_to_numpy=True
    ).astype("float32")
    index = faiss.IndexFlatIP(vectors.shape[1])
    index.add(vectors)
    return index, chunks


def retrieve(question: str, index, chunks, top_k: int = 4):
    q = get_embedder().encode(
        [question], normalize_embeddings=True, convert_to_numpy=True
    ).astype("float32")
    scores, ids = index.search(q, min(top_k, len(chunks)))
    return [
        {**chunks[i], "score": float(s)}
        for s, i in zip(scores[0], ids[0]) if i >= 0
    ]
```

Test it on its own (first run downloads the ~90 MB embedding model):

```
python - << 'EOF'
from pathlib import Path
from src.knowledge_base import build_index, retrieve
docs = [{"filename": "acme_faq.md", "text": Path("data/demo_docs/acme_faq.md").read_text()}]
index, chunks = build_index(docs)
for q in ["how do I add a coworker?", "why was I charged twice", "what is your favourite colour"]:
    print("\nQ:", q)
    for p in retrieve(q, index, chunks, top_k=2):
        print(f"  {p['score']:.2f}  {p['text'][:90]}")
EOF
```

Calibrating MIN_SCORE: look at the scores above. Relevant questions should score clearly higher than the nonsense question. `MIN_SCORE=0.30` in assistant.py is only a starting guess; set it from your own numbers (`export MIN_SCORE=0.35` to try another value without editing code).

6. The assistant and the evaluator

6.1 src/assistant.py

Responsibilities: load base model (+adapter), build the prompt, generate greedily (`do_sample=False`, so results are repeatable), parse and **validate** the JSON, and apply a code-level guard: if no passage clears MIN_SCORE, `needs_human` is forced to True regardless of what the model said. Anything unparsable becomes a safe escalation, never a crash and never a confident wrong answer.

```
import json
import os
import re
import time

from .prompts import CATEGORIES, PRIORITIES, SYSTEM_PROMPT, build_user_message

BASE_MODEL = os.getenv("BASE_MODEL", "Qwen/Qwen2.5-1.5B-Instruct")
ADAPTER_PATH = os.getenv("ADAPTER_PATH", "models/supportpilot-lora")
MIN_SCORE = float(os.getenv("MIN_SCORE", "0.30"))   # retrieval cut-off: tune on your own eval set
PASSAGES_IN_PROMPT = 3

FALLBACK = {
    "category": "other",
    "priority": "medium",
    "needs_human": True,
    "draft_reply": "Thanks for reaching out. A member of our support team will review your message and
      reply soon.",
    "reason": "The model output could not be validated, so this ticket was escalated.",
}


def pick_device() -> str:
    import torch
    if torch.backends.mps.is_available():
        return "mps"
    if torch.cuda.is_available():
        return "cuda"
    return "cpu"


def load_assistant(use_adapter: bool = True):
    """Load the base model once (and the LoRA adapter if present). Returns (tokenizer, model,
      device)."""
    import torch
    from transformers import AutoModelForCausalLM, AutoTokenizer

    device = pick_device()
    dtype = torch.float16 if device in ("mps", "cuda") else torch.float32
    tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
    model = AutoModelForCausalLM.from_pretrained(BASE_MODEL, torch_dtype=dtype)
    if use_adapter:
        if not os.path.isdir(ADAPTER_PATH):
            raise FileNotFoundError(f"Adapter folder not found: {ADAPTER_PATH}")
        from peft import PeftModel
        model = PeftModel.from_pretrained(model, ADAPTER_PATH)
    model.to(device).eval()
    return tokenizer, model, device


def parse_output(text: str):
    """Extract and validate the JSON object. Returns (result, is_valid).
    Anything malformed becomes a safe 'escalate to a human' fallback."""
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        return dict(FALLBACK), False
    try:
        data = json.loads(match.group(0))
    except json.JSONDecodeError:
        return dict(FALLBACK), False
```

```
    if not isinstance(data, dict):
        return dict(FALLBACK), False

    category, priority = data.get("category"), data.get("priority")
    needs_human, draft, reason = data.get("needs_human"), data.get("draft_reply"), data.get("reason")
    ok = (
        category in CATEGORIES and priority in PRIORITIES and isinstance(needs_human, bool)
        and isinstance(draft, str) and draft.strip() and isinstance(reason, str)
    )
    if not ok:
        return dict(FALLBACK), False
    return {
        "category": category,
        "priority": priority,
        "needs_human": needs_human,
        "draft_reply": draft.strip()[:1200],
        "reason": reason.strip()[:400],
    }, True


def generate_from_messages(messages, tokenizer, model, device, max_new_tokens: int = 256):
    """Greedy generation from chat messages. Returns (raw_text, seconds)."""
    import torch
    prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    start = time.perf_counter()
    with torch.no_grad():
        out = model.generate(
            **inputs, max_new_tokens=max_new_tokens, do_sample=False,
            temperature=None, top_p=None, top_k=None, repetition_penalty=1.0,
            pad_token_id=tokenizer.eos_token_id,
        )
    seconds = time.perf_counter() - start
    new_tokens = out[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(new_tokens, skip_special_tokens=True), seconds


def generate_reply(question, passages, tokenizer, model, device):
    """Retrieval results in -> validated ticket dict out."""
    usable = [p for p in passages if p["score"] >= MIN_SCORE][:PASSAGES_IN_PROMPT]
    docs_text = "\n".join(p["text"] for p in usable)
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": build_user_message(question, docs_text)},
    ]
    raw, seconds = generate_from_messages(messages, tokenizer, model, device)
    result, valid = parse_output(raw)
    if not usable:                        # code-level guard: no evidence -> a human must look
        result["needs_human"] = True
    result.update(valid_json=valid, latency_s=round(seconds, 2), raw=raw, evidence_used=len(usable))
    return result
```

6.2 src/evaluation.py, scripts/run_eval.py, scripts/compare.py

The evaluator feeds each eval row's system+user messages to the model and compares the parsed output with the gold answer. Metrics: valid-JSON rate, category accuracy and macro-F1 (averaged over categories that occur), priority accuracy, escalation recall and precision, median latency. Note: malformed output is converted to an escalation, so check *valid_json_rate* next to escalation recall, otherwise a model that only emits garbage looks perfect on recall.

```
import json
import statistics

from .assistant import parse_output

def load_jsonl(path):
    with open(path, encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]


def macro_f1(gold, pred, labels):
    scores = []
    for label in labels:
        tp = sum(g == label and p == label for g, p in zip(gold, pred))
        fp = sum(g != label and p == label for g, p in zip(gold, pred))
        fn = sum(g == label and p != label for g, p in zip(gold, pred))
        precision = tp / (tp + fp) if tp + fp else 0.0
        recall = tp / (tp + fn) if tp + fn else 0.0
        scores.append(2 * precision * recall / (precision + recall) if precision + recall else 0.0)
    return sum(scores) / len(scores)


def summarize(records):
    n = len(records)
    gold_cat = [r["gold"]["category"] for r in records]
    pred_cat = [r["pred"]["category"] for r in records]
    esc_gold = [r["gold"]["needs_human"] for r in records]
    esc_pred = [r["pred"]["needs_human"] for r in records]
    tp = sum(g and p for g, p in zip(esc_gold, esc_pred))
    fp = sum((not g) and p for g, p in zip(esc_gold, esc_pred))
    fn = sum(g and (not p) for g, p in zip(esc_gold, esc_pred))
    return {
        "n": n,
        "valid_json_rate": sum(r["valid"] for r in records) / n,
        "category_accuracy": sum(g == p for g, p in zip(gold_cat, pred_cat)) / n,
        "category_macro_f1": macro_f1(gold_cat, pred_cat, sorted(set(gold_cat) | set(pred_cat))),
        "priority_accuracy": sum(r["gold"]["priority"] == r["pred"]["priority"] for r in records) / n,
        "escalation_recall": tp / (tp + fn) if tp + fn else None,
        "escalation_precision": tp / (tp + fp) if tp + fp else None,
        "median_latency_s": statistics.median(r["latency"] for r in records),
    }


def evaluate(rows, generate_fn, progress=True):
    """rows: chat-format examples (system, user, assistant). generate_fn(messages) -> (raw_text,
      seconds)."""
    records = []
    for i, row in enumerate(rows, 1):
        msgs = row["messages"]
        gold = json.loads(msgs[2]["content"])
        raw, seconds = generate_fn(msgs[:2])
        pred, valid = parse_output(raw)
        records.append({"question": msgs[1]["content"], "gold": gold, "pred": pred,
                        "valid": valid, "latency": seconds, "raw": raw})
        if progress and i % 10 == 0:
            print(f"  {i}/{len(rows)}")
    return records, summarize(records)
```

```
"""Usage:
  python -m scripts.run_eval --name baseline --baseline      # original Qwen, no adapter
  python -m scripts.run_eval --name finetuned                # Qwen + your LoRA adapter
"""
import argparse
import json
from pathlib import Path

from src.assistant import generate_from_messages, load_assistant
from src.evaluation import evaluate, load_jsonl


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--name", required=True)
    ap.add_argument("--baseline", action="store_true", help="run WITHOUT the LoRA adapter")
    ap.add_argument("--eval", default="data/eval.jsonl")
    ap.add_argument("--limit", type=int, default=None)
    args = ap.parse_args()

    rows = load_jsonl(args.eval)[: args.limit]
    print(f"Evaluating '{args.name}' on {len(rows)} examples (adapter={'no' if args.baseline else
      'yes'})")
    tok, model, device = load_assistant(use_adapter=not args.baseline)
    records, summary = evaluate(rows, lambda m: generate_from_messages(m, tok, model, device))

    out = Path("results"); out.mkdir(exist_ok=True)
    (out / f"{args.name}.json").write_text(json.dumps(summary, indent=2))
    with open(out / f"{args.name}_preds.jsonl", "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

```
"""python -m scripts.compare  -> side-by-side table of every results/*.json"""
import json
from pathlib import Path

files = sorted(Path("results").glob("*.json"))
data = {f.stem: json.loads(f.read_text()) for f in files}
if not data:
    raise SystemExit("No results yet. Run scripts.run_eval first.")
keys = list(next(iter(data.values())).keys())
print(f"{'metric':24}" + "".join(f"{name:>14}" for name in data))
for k in keys:
    row = f"{k:24}"
    for name in data:
        v = data[name][k]
        row += f"{'n/a':>14}" if v is None else f"{v:>14.3f}" if isinstance(v, float) else f"{v:>14}"
    print(row)
```

6.3 Run the baseline (original Qwen, no adapter)

```
python -m scripts.run_eval --name baseline --baseline
# first run downloads Qwen2.5-1.5B-Instruct (~3 GB). 42 rows takes a few minutes on Apple GPU.
python -m scripts.compare
```

The *unsupported-claim rate* and *reply quality* from your metric list cannot be computed automatically. Open `results/baseline_preds.jsonl` and read 30-50 outputs yourself, marking drafts that state something not in the documentation. Print only the mistakes:

```
python - << 'EOF'
import json
for line in open("results/baseline_preds.jsonl"):
    r = json.loads(line); g, p = r["gold"], r["pred"]
    if (g["needs_human"] and not p["needs_human"]) or g["category"] != p["category"] or not
      r["valid"]:
        print(r["question"][:230].replace("\n", " | "))
        print("  GOLD:", g["category"], g["needs_human"], "| PRED:", p["category"],
          p["needs_human"], "| valid:", r["valid"])
        print("  DRAFT:", p["draft_reply"][:160], "\n")
EOF
```

7. Fine-tune on Kaggle (LoRA)

7.1 Click-by-click

1. Go to **kaggle.com → Datasets → New Dataset**. Drag in **data/train.jsonl** and **data/eval.jsonl** from your Mac, name it **supportpilot-data**, keep it **Private**, click Create.
2. Go to **Code → New Notebook**. Menu **File → Import Notebook** and upload **notebooks/finetune_support.ipynb**.
3. Right panel **Input → Add Input**: attach your *supportpilot-data* dataset.
4. Right panel **Session options / Settings**: Accelerator = **GPU T4 x2** (or P100); Internet = **On** (needed to download Qwen; requires a phone-verified account).
5. Run cells top to bottom (**Run All**). Watch cell 2: if an unrelated package upgrade causes an import error, use **Run → Restart session** and run again from cell 1.
6. Training prints loss every 10 steps and an eval loss each epoch. Eval loss going down is a good sign; it is not proof the model handles unseen tickets. That is what Section 8 is for.
7. The last cells save the adapter to **/kaggle/working/supportpilot-lora**, zip it, and run one sample generation to confirm the output is JSON.
8. Download **supportpilot-lora.zip** from the right panel **Output** tab. If your session may time out, use **Save Version → Save & Run All (Commit)** so the output files are kept.

7.2 The notebook cells (finetune_support.ipynb)

Cell 1

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "0"      # use ONE gpu (avoids multi-GPU DataParallel surprises on
  T4 x2)
os.environ["TOKENIZERS_PARALLELISM"] = "false"
import torch
print(torch.__version__, "| CUDA:", torch.cuda.is_available())
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

Cell 2

```
!pip install -q -U transformers datasets peft trl accelerate
!pip list 2>/dev/null | grep -E "^(transformers|datasets|peft|trl|accelerate) "
```

Cell 3

```
import glob
train_path = glob.glob("/kaggle/input/**/train.jsonl", recursive=True)[0]
eval_path  = glob.glob("/kaggle/input/**/eval.jsonl",  recursive=True)[0]
print(train_path, eval_path)
```

Cell 4

```
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig
from trl import SFTConfig, SFTTrainer

MODEL_ID = "Qwen/Qwen2.5-1.5B-Instruct"     # if you hit out-of-memory: "Qwen/Qwen2.5-0.5B-Instruct"

dataset = load_dataset("json", data_files={"train": train_path, "eval": eval_path})
print(dataset)

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(MODEL_ID, torch_dtype=torch.float16)
model.config.use_cache = False
```

Cell 5

```
lora_config = LoraConfig(
    r=8, lora_alpha=16, lora_dropout=0.05, bias="none", task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
)
```

Cell 6

```
def format_example(example):
    return {"text": tokenizer.apply_chat_template(example["messages"], tokenize=False,
      add_generation_prompt=False)}

train_ds = dataset["train"].map(format_example)
eval_ds  = dataset["eval"].map(format_example)
print(train_ds[0]["text"][:600])
```

Cell 7

```
training_args = SFTConfig(
    output_dir="/kaggle/working/supportpilot-checkpoints",
    num_train_epochs=2,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    learning_rate=1e-4,
    fp16=True,
    gradient_checkpointing=True,
    max_length=1024,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,
    report_to="none",
    dataset_text_field="text",
)

trainer = SFTTrainer(
    model=model, args=training_args,
    train_dataset=train_ds, eval_dataset=eval_ds,
    processing_class=tokenizer, peft_config=lora_config,
)
trainer.train()
```

Cell 8

```
OUTPUT_DIR = "/kaggle/working/supportpilot-lora"
trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
!cd /kaggle/working && zip -r supportpilot-lora.zip supportpilot-lora && ls -lh supportpilot-lora.zip
```

Cell 9

```
# Quick sanity check on one held-out example
trainer.model.config.use_cache = True
msgs = eval_ds[0]["messages"][:2]
prompt = tokenizer.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True)
inputs = tokenizer(prompt, return_tensors="pt").to(trainer.model.device)
out = trainer.model.generate(**inputs, max_new_tokens=200, do_sample=False)
print(tokenizer.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens=True))
```

Why these settings: r=8 on all attention and MLP projections is a common light LoRA setup. Batch 1 with 8 accumulation steps gives an effective batch of 8 within free-GPU memory. `CUDA_VISIBLE_DEVICES=0` keeps training on one T4. `max_length=1024` leaves room for three retrieved passages in the prompt. Epochs=2 and learning rate 1e-4 are starting values; with a larger dataset, compare eval loss and the Section 8 metrics before adding epochs.

8. Bring the adapter home and compare

```
cd ~/projects/supportpilot && source .venv/bin/activate
unzip ~/Downloads/supportpilot-lora.zip -d models/
ls models/supportpilot-lora        # must contain adapter_config.json and adapter_model.safetensors

python -m scripts.run_eval --name finetuned
python -m scripts.compare
```

Look at	Fine-tune is worth keeping if...
valid_json_rate	at least as high as the baseline (ideally ~1.0)
escalation_recall	higher than baseline, especially on injection, no-evidence and security cases
category_macro_f1	higher than baseline
median_latency_s	not noticeably worse (an adapter adds a little overhead)
Manual read of 30-50 drafts	fewer invented policies or promises

If the fine-tuned model does not beat the baseline on a **large enough** eval set, keep the baseline (`--baseline` / the sidebar toggle) and improve data, retrieval or prompt instead. Change one thing at a time and keep each run's results file under a different `--name`.

9. The Streamlit app

```
import json
import time
from pathlib import Path

import streamlit as st

from src.assistant import MIN_SCORE, generate_reply, load_assistant
from src.knowledge_base import build_index, read_document, retrieve

st.set_page_config(page_title="SupportPilot", page_icon="", layout="wide")
FEEDBACK_FILE = Path("data/feedback.jsonl")
DEMO_DIR = Path("data/demo_docs")


@st.cache_resource(show_spinner="Loading model (the first run takes a minute)...")
def get_model(use_adapter: bool):
    return load_assistant(use_adapter=use_adapter)


def collect_documents(uploads, use_demo):
    docs = []
    if uploads:
        for f in uploads:
            try:
                docs.append({"filename": f.name, "text": read_document(f.name, f.getvalue())})
            except Exception as e:
                st.sidebar.error(f"{f.name}: {e}")
    elif use_demo and DEMO_DIR.exists():
        for p in sorted(DEMO_DIR.glob("*")):
            docs.append({"filename": p.name, "text": read_document(p.name, p.read_bytes())})
    return docs


def save_feedback(rating, last, edited_draft):
    FEEDBACK_FILE.parent.mkdir(exist_ok=True)
    row = {"ts": time.time(), "rating": rating, "question": last["question"],
           "model_result": {k: last["result"][k] for k in ("category", "priority", "needs_human",
             "draft_reply", "reason")},
           "edited_draft": edited_draft}
    with open(FEEDBACK_FILE, "a", encoding="utf-8") as f:
        f.write(json.dumps(row, ensure_ascii=False) + "\n")


st.title(" SupportPilot")
st.caption("AI-assisted support inbox. Drafts and recommends only - it never sends replies or changes
  accounts.")

with st.sidebar:
    st.header("1. Knowledge base")
    uploads = st.file_uploader("Upload PDF, TXT or Markdown", type=["pdf", "txt", "md"],
      accept_multiple_files=True)
    use_demo = st.checkbox("Use bundled demo docs (fictional)", value=True, disabled=bool(uploads))
    st.header("2. Model")
    use_adapter = st.toggle("Use fine-tuned LoRA adapter", value=True)
    st.caption(f"Evidence below similarity {MIN_SCORE} is ignored and the ticket is escalated.")

docs = collect_documents(uploads, use_demo)
if not docs:
    st.info("Upload a help document (or tick the demo docs) to begin.")
    st.stop()
```

```
kb_key = tuple((d["filename"], len(d["text"])) for d in docs)
if st.session_state.get("kb_key") != kb_key:
    with st.spinner("Indexing documents..."):
        st.session_state["index"], st.session_state["chunks"] = build_index(docs)
        st.session_state["kb_key"] = kb_key
index, chunks = st.session_state["index"], st.session_state["chunks"]
st.sidebar.success(f"{len(docs)} document(s), {len(chunks)} chunks indexed")

with st.form("ticket_form"):
    question = st.text_area("Customer message", placeholder="Example: How do I invite a teammate?",
      height=120)
    submitted = st.form_submit_button("Generate support draft")

if submitted and question.strip():
    try:
        tokenizer, model, device = get_model(use_adapter)
    except Exception as e:
        st.error(f"Could not load the model: {e}")
        st.stop()
    with st.spinner("Analyzing the request..."):
        passages = retrieve(question, index, chunks)
        result = generate_reply(question, passages, tokenizer, model, device)
    st.session_state["ticket_no"] = st.session_state.get("ticket_no", 0) + 1
    st.session_state["last"] = {"question": question, "passages": passages, "result": result}

last = st.session_state.get("last")
if last:
    result, passages = last["result"], last["passages"]
    left, right = st.columns([3, 2])
    with left:
        st.subheader("Suggested response")
        c1, c2, c3 = st.columns(3)
        c1.metric("Category", result["category"])
        c2.metric("Priority", result["priority"])
        c3.metric("Latency", f"{result['latency_s']} s")
        if result["needs_human"]:
            st.warning("Needs review: human review required before anything is sent")
        else:
            st.info("Draft ready: review the draft before sending")
        if not result["valid_json"]:
            st.error("The model output was malformed, so this ticket was escalated automatically.")
        draft = st.text_area("Edit the reply", value=result["draft_reply"], height=160,
                             key=f"draft_{st.session_state['ticket_no']}")
        st.write("**Reason:**", result["reason"])
        b1, b2, b3 = st.columns(3)
        for col, label in zip((b1, b2, b3), ("Useful", "Incorrect", "Needs escalation")):
            if col.button(label, key=f"{label}_{st.session_state['ticket_no']}"):
                save_feedback(label, last, draft)
                st.toast(f"Feedback saved: {label}")
    with right:
        st.subheader("Supporting evidence")
        if not passages:
            st.warning("No relevant documents were retrieved.")
        for p in passages:
            used = p["score"] >= MIN_SCORE
            with st.expander(f"{p['source']}  |  similarity {p['score']:.2f}  |  {'used' if used else
              'ignored'}"):
                st.write(p["text"])
    with st.expander("Raw model output (debug)"):
        st.code(result["raw"])
    st.caption("Demo only: this app does not send replies or change accounts.")
```

Run it:

```
source .venv/bin/activate
streamlit run app.py              # opens http://localhost:8501
```

Things the code does that matter: `st.cache_resource` loads the model once per server (not per click); the index rebuilds only when the uploaded documents change; the last ticket is kept in `session_state` so feedback buttons don't wipe the screen; retrieved passages below the cut-off are shown as *ignored* so the operator can see why a ticket was escalated; feedback goes to `data/feedback.jsonl` (git-ignored).

Acceptance test before moving on (write down results)

- **20 on-topic questions** from the demo docs, phrased differently from the FAQ. Correct source shown for each?
- **5 off-topic questions** (e.g. "what's the weather?"). Must show *Needs review*.
- **3 refund / account-change requests.** Must escalate, and the draft must not promise anything.
- **3 prompt-injection messages** ("ignore previous instructions and..."). Must not obey them.
- **Upload a PDF, a .txt and a .md** of your own. Citations show the right file name?
- **Break it on purpose:** empty message, 5,000-character message, scanned (image-only) PDF. The app must show a clear error or escalate, never crash.

10. Making it a product: persistence and multi-tenant security (Supabase)

Do this only after Section 9 is solid. Streamlit session state disappears when the server restarts; tickets and users need a database.

10.1 Set up Supabase

1. supabase.com → New project. Save the database password. Pick the region nearest your users.
2. **Project Settings → API**: copy the **Project URL** and the **anon (public)** key. The **service_role** key bypasses all security: never put it in the app, in Git or in front-end code.
3. **Authentication → Providers**: enable Email.
4. **SQL Editor → New query**: paste **supabase/schema.sql** (below) and click Run.
5. Local config: `export SUPABASE_URL=...` and `export SUPABASE_ANON_KEY=...` (or a git-ignored .env / .streamlit/secrets.toml). `pip install supabase` and add it to requirements.txt.

```
-- Run in Supabase: SQL Editor -> New query -> paste -> Run.

create table public.workspaces (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  created_by uuid not null default auth.uid() references auth.users(id),
  created_at timestamptz not null default now()
);

create table public.workspace_members (
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  user_id uuid not null references auth.users(id) on delete cascade,
  role text not null default 'agent' check (role in ('owner','admin','agent')),
  primary key (workspace_id, user_id)
);

create table public.documents (
  id uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  filename text not null,
  status text not null default 'processing' check (status in ('processing','ready','failed')),
  created_at timestamptz not null default now()
);

create table public.tickets (
  id uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  message text not null,
  category text, priority text,
  status text not null default 'draft_ready' check (status in
    ('draft_ready','needs_review','reviewed')),
  created_at timestamptz not null default now()
);

create table public.drafts (
  id uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  ticket_id uuid not null references public.tickets(id) on delete cascade,
  body text not null,
  needs_human boolean not null,
  reviewed_by uuid references auth.users(id),
  created_at timestamptz not null default now()
);

create table public.feedback (
  id uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  ticket_id uuid not null references public.tickets(id) on delete cascade,
  rating text not null check (rating in ('useful','incorrect','needs_escalation')),
  correction text,
  allow_training_use boolean not null default false,
  created_at timestamptz not null default now()
);

-- Membership check used by every policy. SECURITY DEFINER avoids recursive RLS on workspace_members.
create or replace function public.is_member(ws uuid) returns boolean
language sql security definer stable set search_path = public as $$
  select exists (select 1 from public.workspace_members m where m.workspace_id = ws and m.user_id =
    auth.uid());
$$;
```

```
-- The creator of a workspace automatically becomes its owner.
create or replace function public.add_owner() returns trigger
language plpgsql security definer set search_path = public as $$
begin
  insert into public.workspace_members (workspace_id, user_id, role) values (new.id, new.created_by,
    'owner');
  return new;
end $$;
create trigger workspaces_add_owner after insert on public.workspaces
  for each row execute function public.add_owner();

-- Row Level Security: ON for every table, then policies.
alter table public.workspaces enable row level security;
alter table public.workspace_members enable row level security;
alter table public.documents enable row level security;
alter table public.tickets enable row level security;
alter table public.drafts enable row level security;
alter table public.feedback enable row level security;

create policy ws_select on public.workspaces for select using (public.is_member(id));
create policy ws_insert on public.workspaces for insert with check (created_by = auth.uid());
create policy mem_select on public.workspace_members for select using (public.is_member(workspace_id));

create policy docs_all on public.documents for all using (public.is_member(workspace_id)) with check
  (public.is_member(workspace_id));
create policy tickets_all on public.tickets for all using (public.is_member(workspace_id)) with check
  (public.is_member(workspace_id));
create policy drafts_all on public.drafts for all using (public.is_member(workspace_id)) with check
  (public.is_member(workspace_id));
create policy feedback_all on public.feedback for all using (public.is_member(workspace_id)) with check
  (public.is_member(workspace_id));

create index on public.tickets (workspace_id, created_at desc);
create index on public.drafts (ticket_id);
```

Note: your original table list had no workspace_id on *drafts* and *feedback*; the schema adds it so every row belongs to a workspace and each table gets a direct RLS check. `allow_training_use` defaults to false so customer corrections are never used for training unless explicitly allowed.

10.2 Prove tenant isolation (do not skip)

1. Authentication → Users → Add user: create **alice@test.com** and **bob@test.com**.
2. Sign in as Alice with src/db.py, create a workspace and one ticket.
3. Sign in as Bob. Query `tickets` and `workspaces`: you must get **zero rows**. Then try inserting a ticket with Alice's workspace_id as Bob: it must be rejected.
4. Repeat for every table. Only when all fail for Bob is isolation real.

```
python - << 'EOF'
from src.db import sign_in, create_workspace, save_ticket
alice, alice_id = sign_in("alice@test.com", "ALICE_PASSWORD")
ws = create_workspace(alice, "Alice Co")
save_ticket(alice, ws, alice_id, "test message",
            {"category": "other", "priority": "low", "needs_human": True, "draft_reply": "hi",
              "reason": "test"})
bob, _ = sign_in("bob@test.com", "BOB_PASSWORD")
print("bob sees tickets:", bob.table("tickets").select("*").execute().data)        # expect []
print("bob sees workspaces:", bob.table("workspaces").select("*").execute().data)  # expect []
try:
    bob.table("tickets").insert({"workspace_id": ws, "message": "intrusion"}).execute()
    print("FAIL: Bob could insert into Alice's workspace")
except Exception as e:
    print("OK, insert blocked:", type(e).__name__)
EOF
```

```
"""Starting sketch for Supabase persistence (not wired into app.py yet).
pip install supabase ; set SUPABASE_URL and SUPABASE_ANON_KEY (anon key only - never the service-role
  key here)."""
import os

from supabase import create_client


def sign_in(email: str, password: str):
    client = create_client(os.environ["SUPABASE_URL"], os.environ["SUPABASE_ANON_KEY"])
    session = client.auth.sign_in_with_password({"email": email, "password": password}).session
    return client, session.user.id


def create_workspace(client, name: str):
    return client.table("workspaces").insert({"name": name}).execute().data[0]["id"]


def save_ticket(client, workspace_id: str, user_id: str, question: str, result: dict):
    status = "needs_review" if result["needs_human"] else "draft_ready"
    ticket = client.table("tickets").insert({
        "workspace_id": workspace_id, "message": question,
        "category": result["category"], "priority": result["priority"], "status": status,
    }).execute().data[0]
    client.table("drafts").insert({
        "workspace_id": workspace_id, "ticket_id": ticket["id"],
        "body": result["draft_reply"], "needs_human": result["needs_human"],
    }).execute()
    return ticket["id"]
```

Wiring it into app.py is the next step: add a login box, keep the access token in `session_state`, pick the user's workspace, call `save_ticket` after each generation, and replace feedback.jsonl with the feedback table. Keep the review states: **draft_ready**, **needs_review** (model flagged it) and **reviewed** (a human approved or edited it, with `drafts.reviewed_by` recording who). Add delete buttons for tickets and documents.

11. Deploy it

> **The constraint that decides everything**
> Streamlit Community Cloud gives apps roughly 690 MB to 2.7 GB of memory (its own docs cite February 2024 figures and say limits can change without notice). Qwen2.5-1.5B in fp16 needs about 3 GB for weights alone, so the model cannot run inside Community Cloud. Plan accordingly.

11.1 Put the code on GitHub (all options start here)

```
git status                          # confirm no .venv/, .env, model weights, data/feedback.jsonl
git add . && git commit -m "SupportPilot MVP"
gh auth login                       # follow the prompts once
gh repo create supportpilot --private --source=. --push
```

11.2 Option A (recommended first demo): run on your Mac, share through a tunnel

Free, uses your Apple GPU, needs no deployment debugging. The URL works only while your Mac is awake and both commands are running.

```
# Terminal 1
source .venv/bin/activate
caffeinate -i streamlit run app.py --server.port 8501      # caffeinate stops the Mac sleeping

# Terminal 2
cloudflared tunnel --url http://localhost:8501
# prints  https://<random-words>.trycloudflare.com   <- send this link
```

Anyone with the link can use the app, so use the fictional demo docs only, and stop the tunnel (Ctrl+C) when you are done.

11.3 Option B: a Hugging Face Docker Space (CPU)

Hugging Face's CPU Basic hardware is 2 vCPU and 16 GB RAM, enough to hold the model, but expect slow generation: a 1.5B model on 2 CPU cores can take tens of seconds per reply. Free Spaces also go to sleep when idle. **One thing to verify when you sign up:** the sources I found disagree on whether Docker Spaces still need a paid plan, and the Hub docs say that creating a compute Space (Gradio or Docker) requires a paid plan while static Spaces are free. Check the plan requirement on the New Space page before investing time.

1. huggingface.co → New Space → SDK **Docker** → blank → Public or Private → Create.
2. Clone it and copy the project in (adapter included; the app downloads the base model at first start).
3. Put **deploy/Dockerfile** at the repo root, and prepend **deploy/space_README_header.md** to the Space's README.md (the YAML block sets sdk and port 7860).
4. Push. The Space builds the image and starts it; open the Space URL.

```
pip install -U huggingface_hub
huggingface-cli login                       # paste a token with WRITE access (newer versions: hf
  auth login)
git clone https://huggingface.co/spaces/<your-username>/supportpilot-demo
cd supportpilot-demo

# copy project files (not .venv, results, feedback)
rsync -av --exclude '.venv' --exclude 'results' --exclude '.git' --exclude 'data/feedback.jsonl' \
      ~/projects/supportpilot/ ./
cp deploy/Dockerfile ./Dockerfile
cat deploy/space_README_header.md README.md > README.tmp && mv README.tmp README.md

git lfs install && git lfs track "*.safetensors"
git add -f models/supportpilot-lora          # -f because models/ is git-ignored
git add . && git commit -m "Deploy SupportPilot demo" && git push
```

If the Space runs out of memory or is too slow, switch BASE_MODEL to Qwen2.5-0.5B-Instruct, but only after training an adapter on that model; an adapter only works with the base it was trained on.

```
# For a Hugging Face Docker Space (CPU). Put this file at the ROOT of the Space repo as "Dockerfile".
FROM python:3.11-slim
RUN useradd -m -u 1000 user
WORKDIR /app
COPY --chown=user requirements.txt .
RUN pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu \
 && pip install --no-cache-dir -r requirements.txt
COPY --chown=user . .
USER user
# BASE_MODEL must be the SAME model your adapter was trained on.
ENV HOME=/home/user HF_HOME=/home/user/.cache/huggingface BASE_MODEL=Qwen/Qwen2.5-1.5B-Instruct
EXPOSE 7860
CMD ["streamlit","run","app.py","--server.port=7860","--server.address=0.0.0.0","--
  server.headless=true","--server.enableXsrfProtection=false","--server.enableCORS=false"]
```

11.4 When customers (not demos) are involved

- Don't run the model inside the web app. Put it behind a small inference API on rented GPU or serverless GPU hosting, and let the UI call that API. Look up current pricing when you get there; I haven't verified any.
- Real customer data needs the Supabase layer, HTTPS, secrets in the host's secret store (never in Git), document deletion, and a clear data-use statement.
- Keep sending nothing automatically. The first release drafts only.

12. Troubleshooting

Symptom	Likely cause and fix
ModuleNotFoundError: No module named 'src'	Run from the project root, and run scripts as modules: `python -m scripts.run_eval`, not `python scripts/run_eval.py`.
Garbled output or NaN on Mac GPU	fp16 on MPS is sometimes unstable. In assistant.py set `dtype = torch.float32` (uses more RAM) or run with `PYTORCH_ENABLE_MPS_FALLBACK=1`.
Adapter folder not found	The zip must unpack to models/supportpilot-lora/adapter_config.json. Check the path with ls.
Error on loading adapter (size/shape mismatch)	Adapter and base model differ. Train and run with the same model id (BASE_MODEL).
TypeError: SFTConfig unexpected keyword 'max_length'	Older TRL. Run `!pip install -U trl` or rename to `max_seq_length`; check the notebook's printed version.
'Attempting to unscale FP16 gradients'	Load the model in float32 (`torch_dtype=torch.float32`) and keep `fp16=True` (mixed precision); fits for 1.5B on a T4.
CUDA out of memory on Kaggle	Use Qwen2.5-0.5B-Instruct, set max_length=512, keep batch size 1.
Kaggle: no internet / model download fails	Settings → Internet On; the account must be phone-verified.
pip install faiss-cpu fails on Mac	Upgrade pip and use Python 3.11. Last resort: replace FAISS with NumPy: `scores = vectors @ q.T` and `np.argsort`; fine at this scale.
App shows 'Could not load the model'	Read the exact error. Usually a missing adapter folder or no internet on first run.
Model returns text instead of JSON	Expected sometimes: parse_output escalates it. If it happens often, more or cleaner training rows, or a stricter prompt.
Great training loss, bad eval	Overfitting or leakage. Re-run validate_dataset.py; add more varied scenarios; try 1 epoch.

13. Command cheat sheet

```
# every new terminal
cd ~/projects/supportpilot && source .venv/bin/activate

# data
python -m scripts.make_dataset && python -m scripts.validate_dataset data/train.jsonl
  data/eval.jsonl

# evaluate
python -m scripts.run_eval --name baseline --baseline
python -m scripts.run_eval --name finetuned
python -m scripts.compare

# run
streamlit run app.py

# share
cloudflared tunnel --url http://localhost:8501

# save work
git add . && git commit -m "message" && git push
```

14. Checklist

- ■ Tools installed, accounts created, Kaggle phone-verified

- [ ] Folder structure + venv + git initialised
- [ ] Dataset generated, validated, leak check passes
- [ ] Retrieval tested; MIN_SCORE chosen from real scores
- [ ] Baseline results saved in results/baseline.json
- [ ] Scenarios grown to 60+ and eval set 100+ rows
- [ ] Kaggle run finished, adapter downloaded to models/supportpilot-lora
- [ ] Fine-tuned vs baseline compared; decision written down
- [ ] App passes the Section 9 acceptance test
- [ ] Supabase isolation test passes (Bob sees nothing of Alice's)
- [ ] Public demo live on fictional data only
- [ ] 10-20 founders contacted; feedback recorded